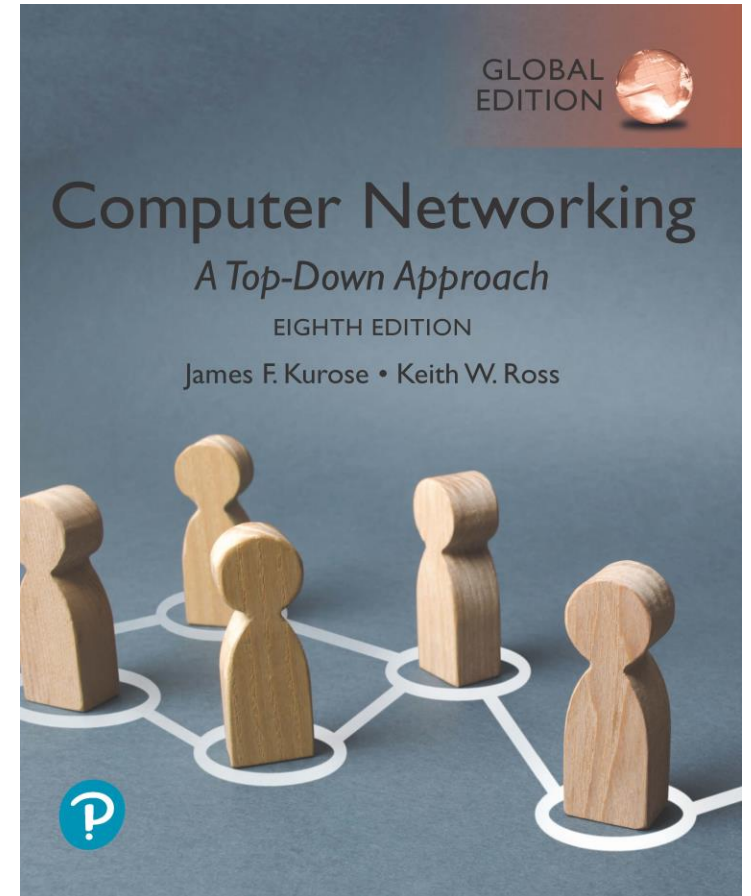


Chapter 6

The Link Layer and LANs



Computer Networking: A Top-Down Approach

8th Edition, Global Edition

Jim Kurose, Keith Ross

Copyright © 2022 Pearson Education Ltd

Link layer and LANs: our goals

- understand principles behind link layer services:
 - error detection, correction
 - sharing a broadcast channel: multiple access
 - link layer addressing
 - local area networks: Ethernet
- instantiation, implementation of various link layer technologies



Link layer, LANs: roadmap

- introduction
- error detection, correction
- multiple access protocols
- LANs
 - addressing, ARP
 - Ethernet
 - switches
- a day in the life of a web request

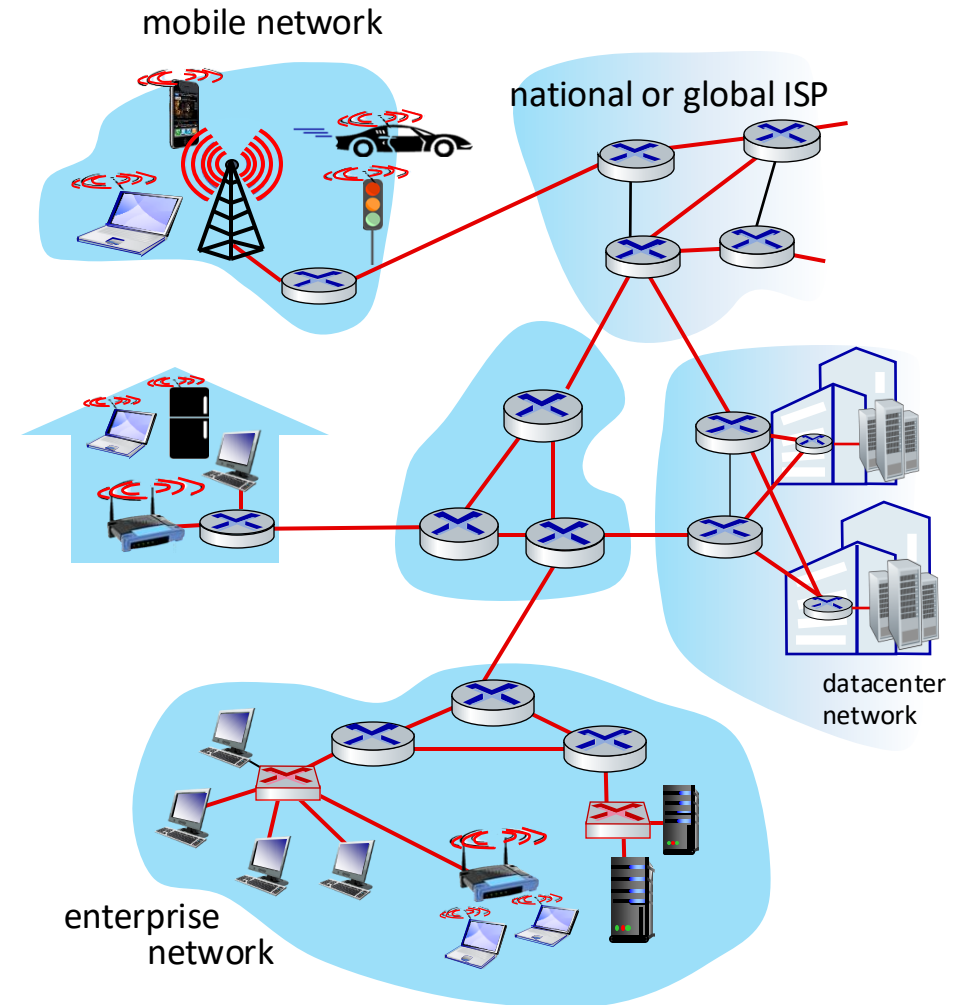


Link layer: introduction

terminology:

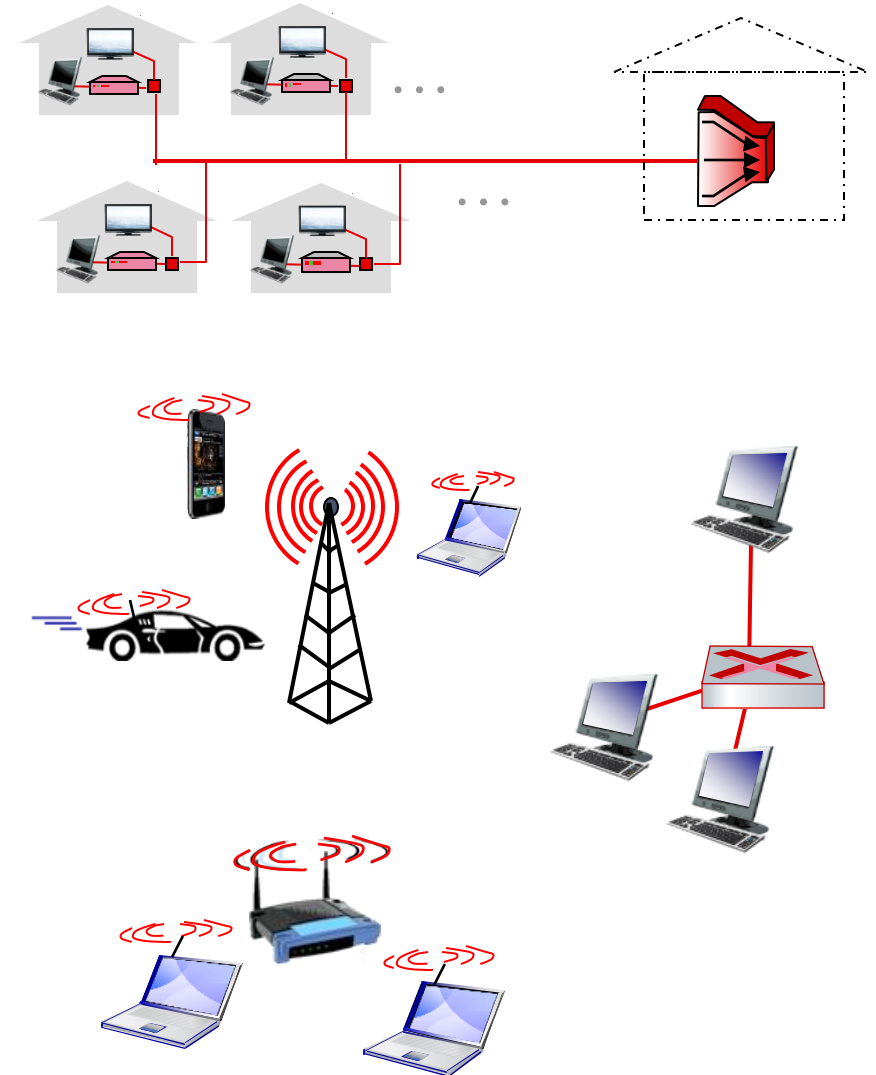
- hosts and routers: nodes
- communication channels that connect adjacent nodes along communication path: links
 - wired
 - wireless
 - LANs
- layer-2 packet: frame, encapsulates datagram

*link layer has responsibility of transferring datagram from one node to **physically adjacent** node over a link*



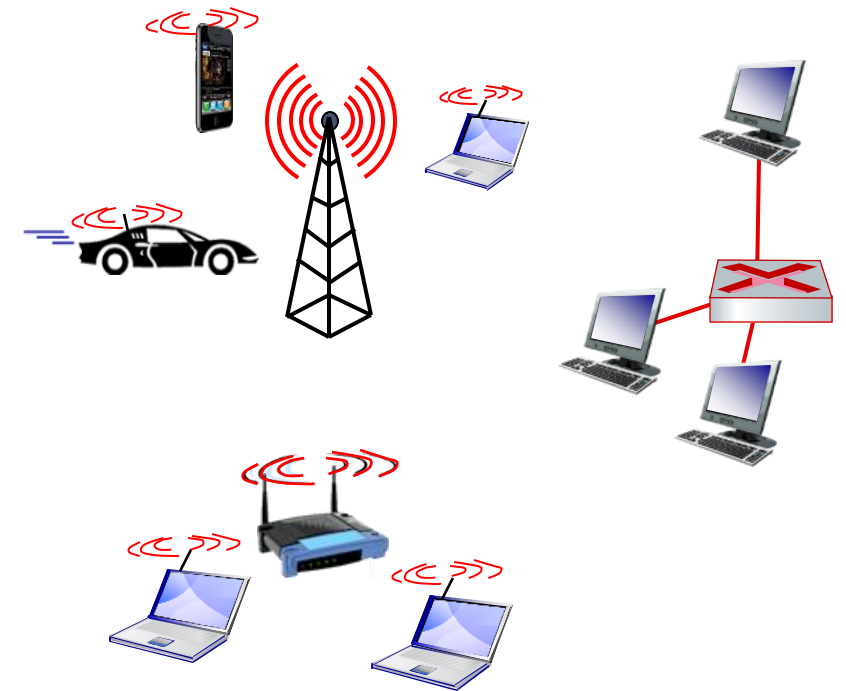
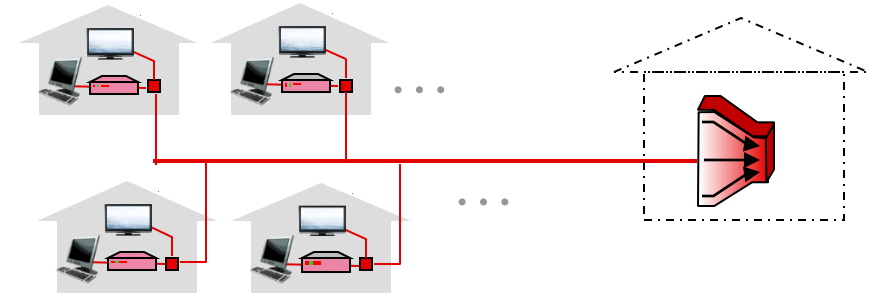
Link layer: services

- **framing, link access:**
 - encapsulate datagram into frame, adding header, trailer
 - channel access if shared medium
 - “MAC” addresses in frame headers identify source, destination (different from IP address!)
- **reliable delivery between adjacent nodes**
 - But we already know how to do this ???
 - seldom used on low bit-error links
 - wireless links: high error rates
 - Q: why both link-level and end-end reliability?



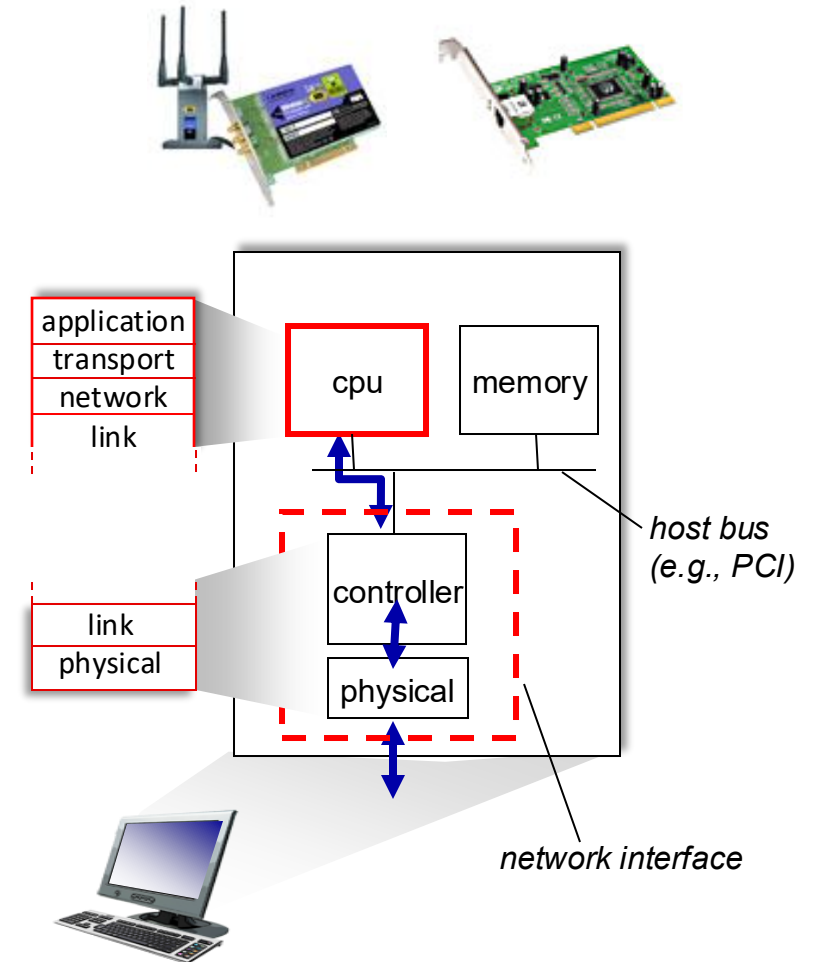
Link layer: services (more)

- **flow control:**
 - **pacing** between adjacent sending and receiving nodes
- **error detection:**
 - errors caused by signal attenuation, noise.
 - receiver detects errors, signals retransmission, or drops frame
- **error correction:**
 - receiver identifies *and corrects* bit error(s) without retransmission
- **half-duplex and full-duplex:**
 - with half duplex, nodes at both ends of link can transmit, but not at same time



Where is the link layer implemented?

- in each-and-every host
- link layer implemented in *network interface card* (NIC) or on a chip
 - Ethernet, WiFi card or chip
 - implements link, physical layer
- attaches into host's system buses
- combination of hardware, software, firmware



Link layer transmits frames over a physical medium

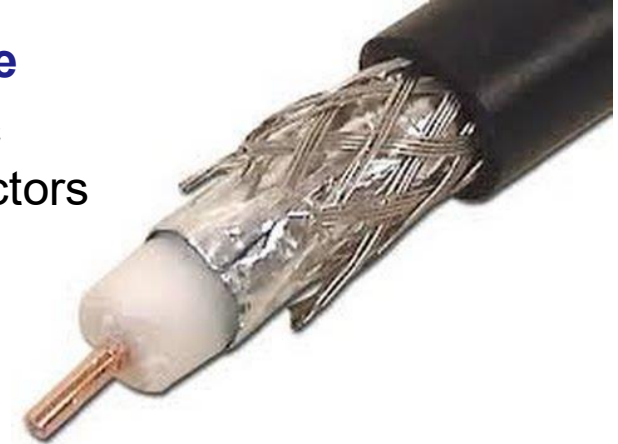
1. Twisted pair

- two insulated copper wires



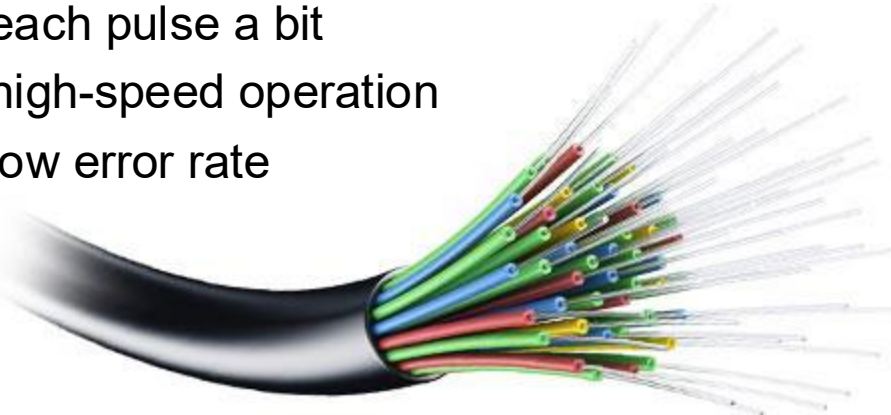
2. Coaxial cable

- two concentric copper conductors



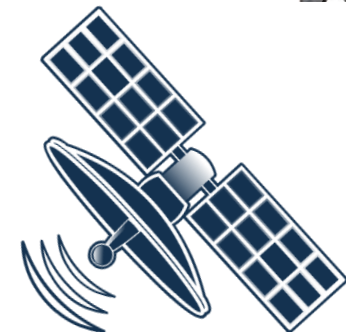
3. Fiber optic cable

- glass fiber carrying light pulses, each pulse a bit
- high-speed operation
- low error rate



4. Radio link

- terrestrial microwave
- wireless LAN
- wide-area (e.g. cellular)
- satellite



Link layer, LANs: roadmap

- introduction
- **error detection, correction**
- multiple access protocols
- LANs
 - addressing, ARP
 - Ethernet
 - switches

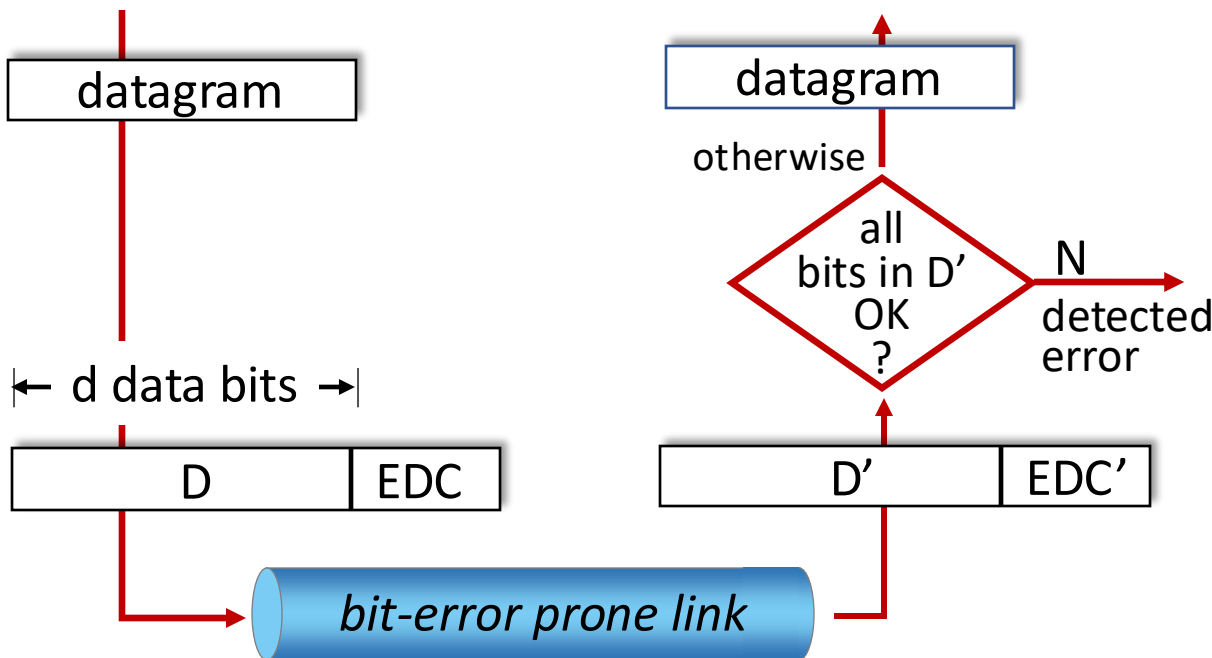


- a day in the life of a web request

Error detection

EDC: error detection and correction bits (e.g., redundancy)

D: data protected by error checking, may include header fields



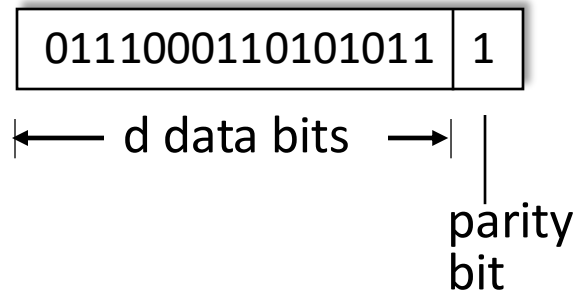
Error detection not 100% reliable!

- protocol may miss some errors, but rarely
- larger EDC field yields better detection and correction

Parity checking

single bit parity:

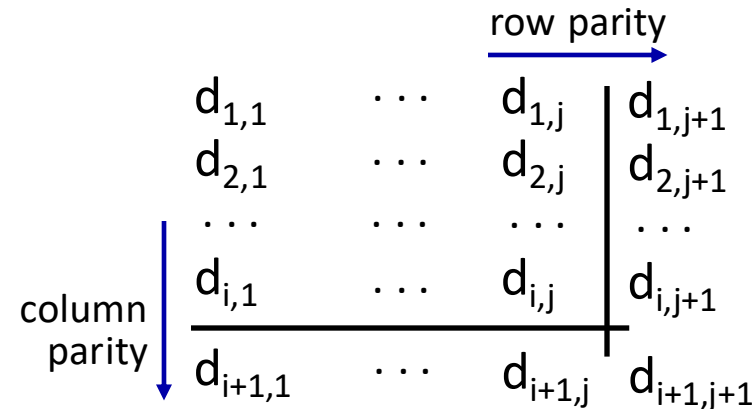
- detect single bit errors



Even parity: set parity bit so there is an even number of 1's

two-dimensional bit parity:

- detect **and correct** single bit errors



no errors:

1	0	1	0	1	1
1	1	1	1	0	0
0	1	1	1	0	1
0	0	1	0	1	0

detected and correctable single-bit error:

1	0	1	0	1	1
1	0	1	1	0	0
0	1	1	1	0	1
0	0	1	0	1	0

Red arrows point to the error in the second row, second column, and the resulting parity errors in the second row and second column.

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Internet checksum (revisit..)

Goal: detect errors (*i.e.*, flipped bits) in transmitted segment

sender:

- treat contents of UDP segment (including UDP header fields and IP addresses) as sequence of 16-bit integers
- **checksum:** addition (one's complement sum) of segment content
- checksum value put into UDP checksum field

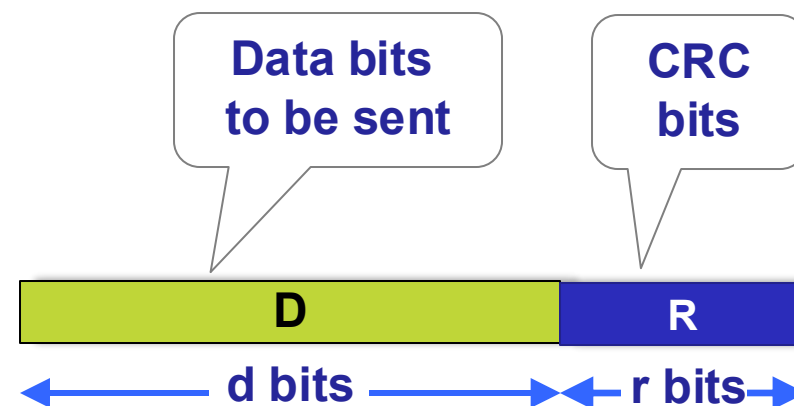
receiver:

- compute checksum of received segment
- check if computed checksum equals checksum field value:
 - not equal - error detected
 - equal - no error detected. *But maybe errors nonetheless?* More later

Error detection

Cyclic Redundancy Check (CRC) – more complex, more robust

- CRC of r bits can detect
 - all burst errors $< r+1$ bits
 - all odd number of bit errors
 - burst error $> r+1$ detected with probability $1 - 0.5^r$
 - **RIKTIG ER:** burst error $> r+1$ detected with probability $1 - (1/2)^r$



Send $\langle D, R \rangle$

Cyclic codes are not only **simple to implement** but have the benefit of being particularly well suited for the detection of **burst errors**: contiguous sequences of erroneous data symbols in messages.

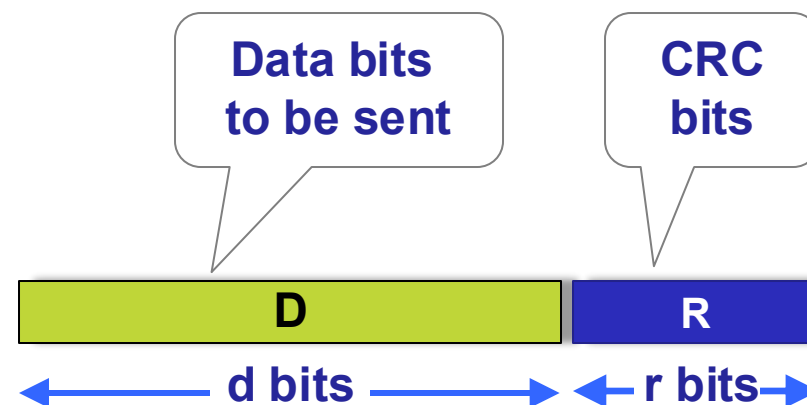
This is important because burst errors are common transmission errors in many communication channels, including magnetic and optical storage devices.

Error detection

Cyclic Redundancy Check (CRC) – more complex, more robust

- View **data bits**, **D**, as a binary number
- Choose a **generator G** = **r+1** bits
 - first and last bit = 1
 - Generator bits are one more than the CRC bits
- Find r **CRC-bits**, **R**, such that
 - **<D,R>** exactly divisible by **G** (modulo 2)

R = is the "remainder" which must be added to D in order to achieve this **<D,R>** characteristics



Send < D,R >

- Receiver knows **G**, divides **<D,R>** by **G**
- **If non-zero remainder: error detected!**



Modulo-2 binær divisjon er en matematisk operasjon som brukes mye innen informatikk og datakommunikasjon, spesielt for å beregne **Cyclic Redundancy Checks (CRC)**.

Selv om det ligner på vanlig divisjon, er det en viktig forskjell: Vi bruker ikke lån eller mente. Operasjonen er utelukkende basert på **XOR-logikk** (Exclusive OR).

Hovedreglene for Modulo-2

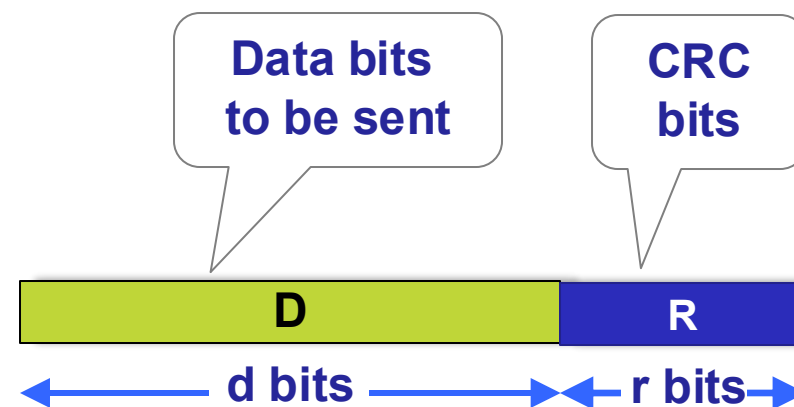
Når vi utfører modulo-2 divisjon, følger vi disse enkle reglene for hver bit-operasjon:

Bit A	Bit B	Resultat (A XOR B)
0	0	0
0	1	1
1	0	1
1	1	0

Cyclic Redundancy Check (CRC) – more complex, more robust

generator G kan oppgis
binært eller som et polynom

$$\begin{array}{c} x^3 + x + 1 \\ | \quad | \quad | \\ 1 \ 0 \ 1 \ 1 \end{array}$$



- Widely used, eg CRC16 (802.11 WiFi) CRC32 (802.3)

Send $\langle D, R \rangle$

CRC16-CCITT $x^{16} + x^{12} + x^5 + 1$

CRC32 $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$

R = is the "remainder" which must be added to D in order to achieve this $\langle D, R \rangle$ characteristics

- $\langle D, R \rangle$ exactly divisible by G (modulo 2)

CRC bits = R = remainder

$$\frac{[D \cdot 2^r]}{G}$$

Multiplikasjon av et binært tall med 2^r er det samme som å right "padde" med r bits satt til 0 fra høyre

Eks

1011100 som er 92 dec

1011100000 som er 736 dec ($=92 \cdot 2^3 = 92 \cdot 8$)

Vi "padder" altså med r bit satt til 0 før vi gjør divisjonen med generator (G)

$\langle D, R \rangle$ satt sammen gir da rest 0 ved divisjon med samme generator (G)

Exercise: CRC-4 find <D,R>

11010011101100

This is first padded with zeros corresponding to the bit length n of the CRC. Here is the first calculation for computing a 3-bit CRC:

```
11010011101100 000 <--- input right padded by 3 bits
1011                <--- divisor (4 bits) =  $x^3 + x + 1$ 
-----
01100011101100 000 <--- result
```

Hva er generatoren G ?

The algorithm acts on the bits directly above the divisor in each step. The result for that iteration is the bitwise XOR of the polynomial divisor with the bits above it. The bits not above the divisor are simply copied directly below for that step. The divisor is then shifted one bit to the right, and the process is repeated until the divisor reaches the right-hand end of the input row. Here is the entire calculation:

```
11010011101100 000 <--- input right padded by 3 bits
1011                <--- divisor
01100011101100 000 <--- result (note the first four bits are the XOR with the divisor beneath, the rest of the bits are unchanged)
1011                <--- divisor ...
00111011101100 000
1011
00010111101100 000
1011
00000001101100 000 <--- note that the divisor moves over to align with the next 1 in the dividend (since quotient for that step was zero)
1011                (in other words, it doesn't necessarily move one bit per iteration)
00000000110100 000
1011
00000000011000 000
1011
00000000001110 000
1011
00000000000101 000
101 1
-----
00000000000000 100 <--- remainder (3 bits). Division algorithm stops here as dividend is equal to zero.
```

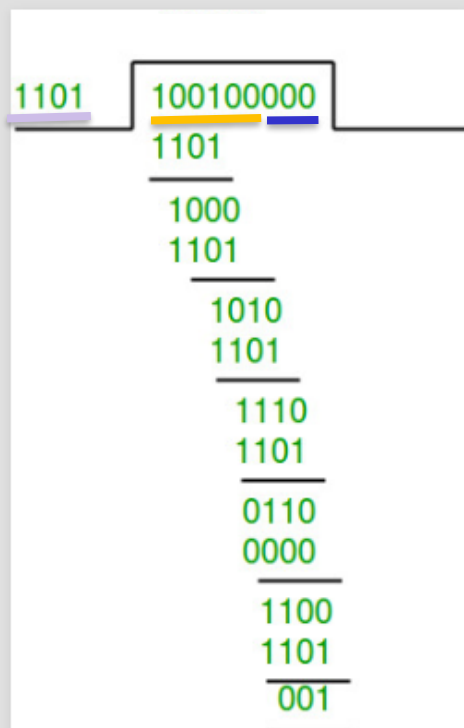
Hva ble R ?

Example – No error in transmission (eksempel hentet fra nett)

Data word to be sent - 100100

Key - 1101 [Or generator polynomial $x^3 + x^2 + 1$]

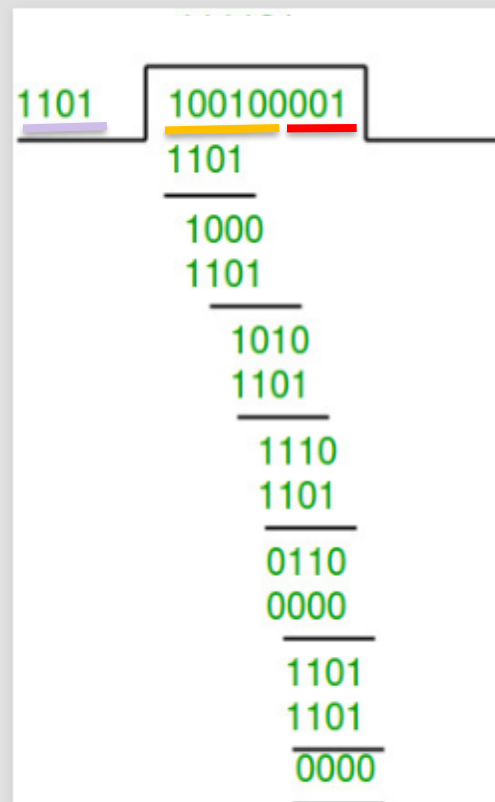
Sender Side:



Therefore, the remainder is 001 and hence the encoded data sent is 100100001.

Receiver Side:

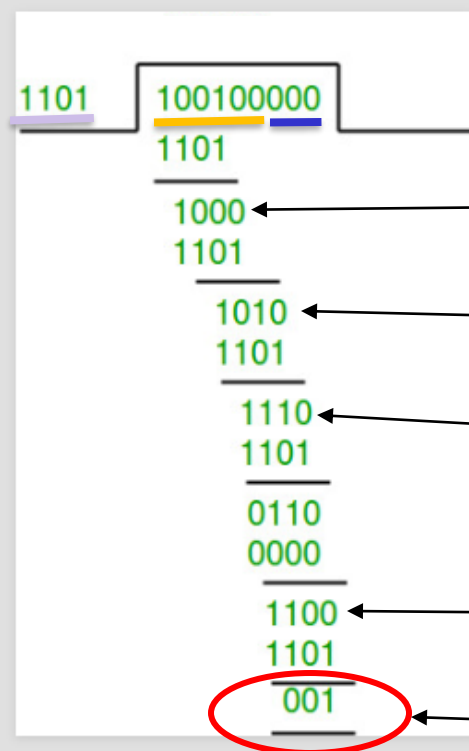
Code word received at the receiver side 100100001



Therefore, the remainder is all zeros. Hence, the data received has no error.

Key - 1101 [Or generator polynomial $x^3 + x^2 + 1$]

Sender Side:



Therefore, the remainder is 001 and hence the encoded data sent is 100100001.

Foreleser sin måte å regne ut/visе på...

D									G
1	0	0	1	0	0	0	0	0	1101
1	1	0	1						
0	1	0	0						
— 1	0	0	0	0	0	0	0	0	
	1	1	0	1					
—		1	0	1	0	0	0	0	
		1	1	0	1				
—			1	1	1	0	0	0	
			1	1	0	1			
—				1	1	0	0		
				1	1	0	1		
—					1				

.....som Bjørn synes er litt ryddigere

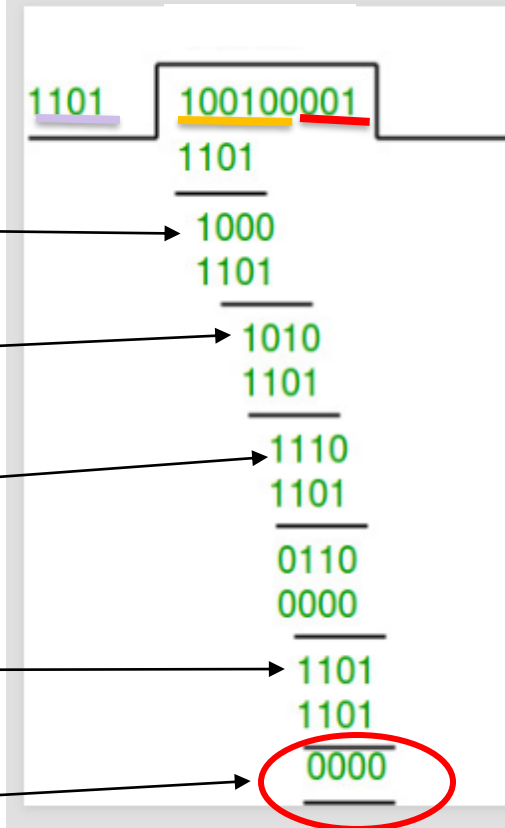
Example – No error in transmission

Foreleser sin måte å regne ut på...

D										G
1	0	0	1	0	0	0	0	1		1101
1	1	0	1							
	1	0	0	0	0	0	0	1		
	1	1	0	1						
		1	0	1	0	0	0	1		
		1	1	0	1					
			1	1	1	0	0	1		
			1	1	0	1				
				1	1	0	1			
					0					

Receiver Side:

Code word received at the receiver side 100100001



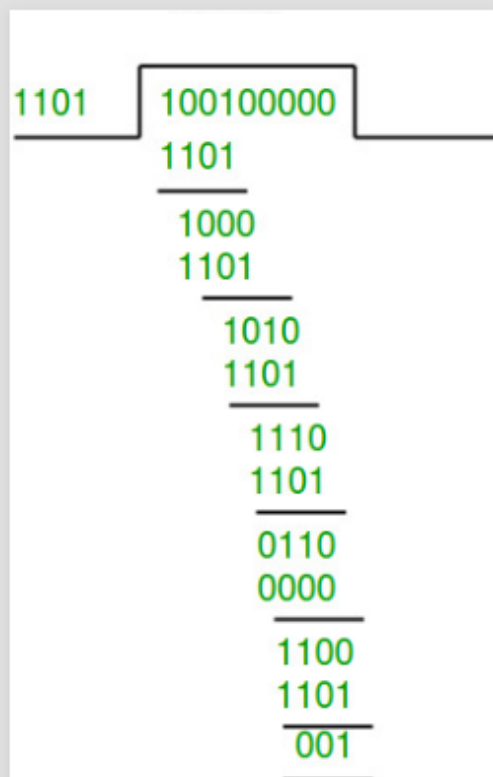
Therefore, the remainder is all zeros. Hence, the data received has no error.

Example – **Error** in transmission

Data word to be sent - 100100

Key - 1101

Sender Side:

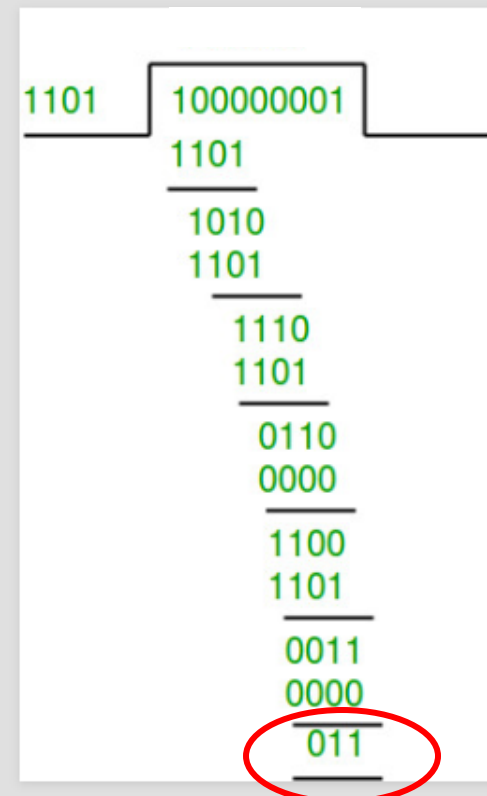


Therefore, the remainder is 001 and hence the code word sent is 1001000001.

Receiver Side

Let there be error in transmission media

Code word received at the receiver side - 100000001



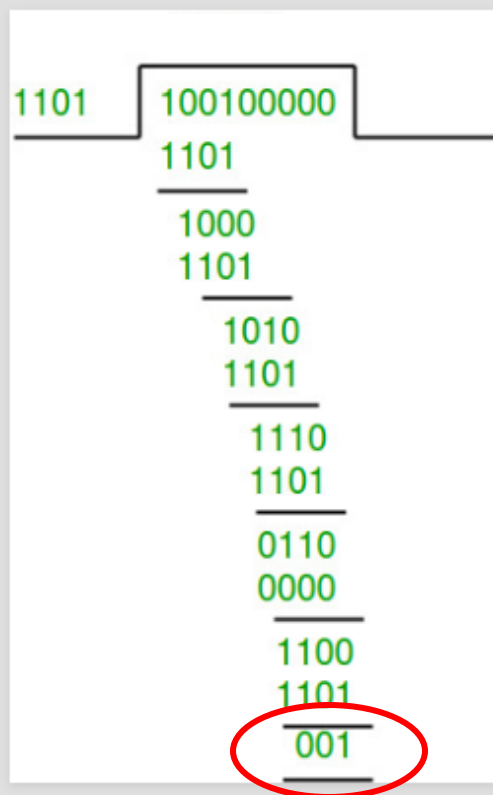
Since the remainder is not all zeroes, the error is detected at the receiver side.

Example – **Error** in transmission

Data word to be sent - 100100

Key - 1101

Sender Side:



Therefore, the remainder is 001 and hence the code word sent is 1001000001.

Foreleser sin måte å regne ut på...

D																G
1	0	0	1	0	0	0	0	0								1101
1	1	0	1													
0	1	0	0													
	1	0	0	0	0	0	0	0								
	1	1	0	1												
		1	0	1	0	0	0	0								
		1	1	0	1											
			1	1	1	0	0	0								
			1	1	0	1										
				1	1	0	0									
				1	1	0	1									
					1											
						1										

Example – **Error** in transmission

Foreleser sin måte å regne ut på...

D																		G
1	0	0	0	0	0	0	0	0	1									1101
1	1	0	1															
	1	0	1	0	0	0	0	1										
	1	1	0	1														
		1	1	1	0	0	0	1										
		1	1	0	1													
			1	1	0	0	1											
			1	1	0	1												
				1	1													
					1	1												

Receiver Side

Let there be error in transmission media

Code word received at the receiver side - 100000001

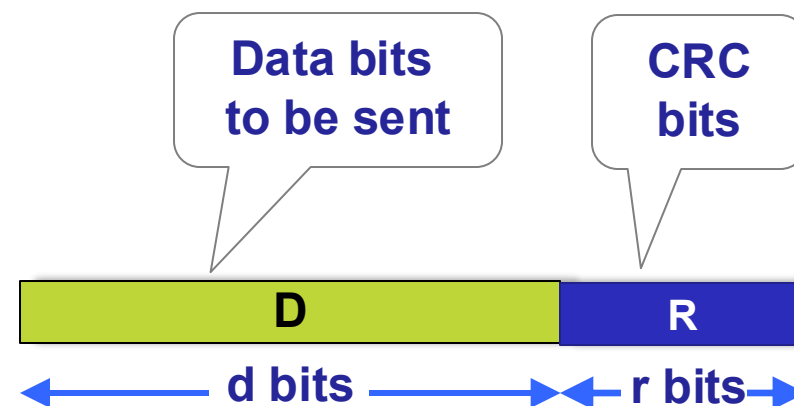
1101	100000001
	1101
	1010
	1101
	1110
	1101
	0110
	0000
	1100
	1101
	0011
	0000
	011

Since the remainder is not all zeroes, the error is detected at the receiver side.

Error detection

Cyclic Redundancy Check (CRC) – more complex, more robust

- CRC of r bits can detect
 - all burst errors $< r+1$ bits
 - all odd number of bit errors
 - burst error $> r+1$ detected with probability $1-0.5^r$



Send $\langle D, R \rangle$

Cyclic codes are not only **simple to implement** but have the benefit of being particularly well suited for the detection of **burst errors**: contiguous sequences of erroneous data symbols in messages.

This is important because burst errors are common transmission errors in many communication channels, including magnetic and optical storage devices.